

ARCHITECTURE BIBLE – SYSTEM, MODEL & DEPLOYMENT SPECIFICATION

The complete architecture of StockSense

Every layer, module, data contract, and deployment detail behind the pipeline that turns five years of OHLCV data and daily news into a SHAP-explained UP/DOWN prediction.

ARCHITECTURE STYLE

8-layer, two-stage ML pipeline

CORE MODELS

LSTM (Keras) → XGBoost → SHAP

SERVING

Streamlit, single-process, local/CPU

Table of Contents

01	System Overview	03
02	High-Level Architecture Diagram	04
03	Layered Architecture Breakdown	06
04	Data Flow — Sequence Walkthrough	08
05	LSTM Model Architecture	09
06	XGBoost Classifier Configuration	10
07	Fused Feature Schema	11
08	Module & Folder Structure	12
09	Interface Contracts	13
10	Deployment Architecture	14
11	Technology Stack Matrix	15
12	Error Handling & Retraining	16
13	Scalability & Performance	17
14	Security & Data Considerations	17
15	Appendix — Full File Tree	18

01 System Overview

StockSense is a two-stage machine learning pipeline wrapped in eight functional layers: it learns a price representation, reads sentiment independently, fuses both with technical indicators, classifies direction, explains the classification, and serves all of it through one dashboard.

Architecturally, the system is a linear, single-direction pipeline — not a distributed or microservice system. Every stage is a Python module operating over Pandas DataFrames, chained sequentially from raw data ingestion to explained output. This is intentional: at the data scale involved (single-ticker, 5-year daily OHLCV plus daily headlines), a monolithic pipeline running on one machine is simpler to build, debug, and reproduce than a distributed system, while still cleanly separating concerns at the module boundary.

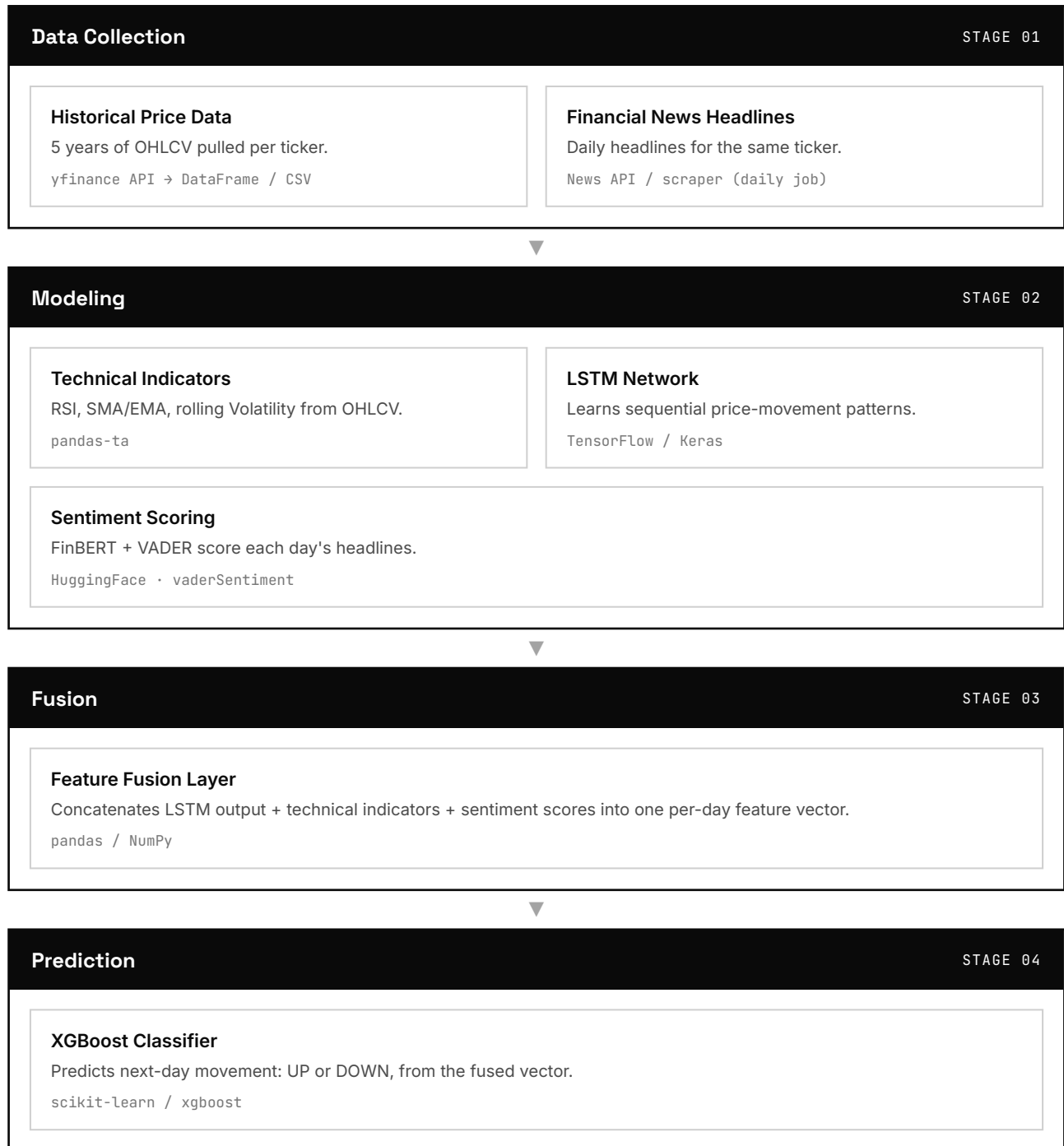
8	2	1	Daily
ARCHITECTURAL LAYERS	TRAINED MODELS	SERVING PROCESS	PREDICTION CADENCE

1.1 Architectural style

- **Pipeline / pipe-and-filter** — each layer takes the previous layer's DataFrame or array output as its input; no layer reaches back into an earlier one.
- **Two-stage modelling** — a sequence model (LSTM) produces a learned representation; a tabular classifier (XGBoost) makes the final call on the fused vector. See Section 05–06 for why this split, not one end-to-end network.
- **Explainability as a first-class stage** — SHAP is not a debugging afterthought; it is layer 7 of 8, sitting between prediction and presentation.
- **Thin presentation layer** — Streamlit (layer 8) contains no modelling logic; it only renders what layers 1–7 already computed.

02 High-Level Architecture Diagram

Six stages, eight components — raw market data and text in on the left, an explained prediction out on the right.



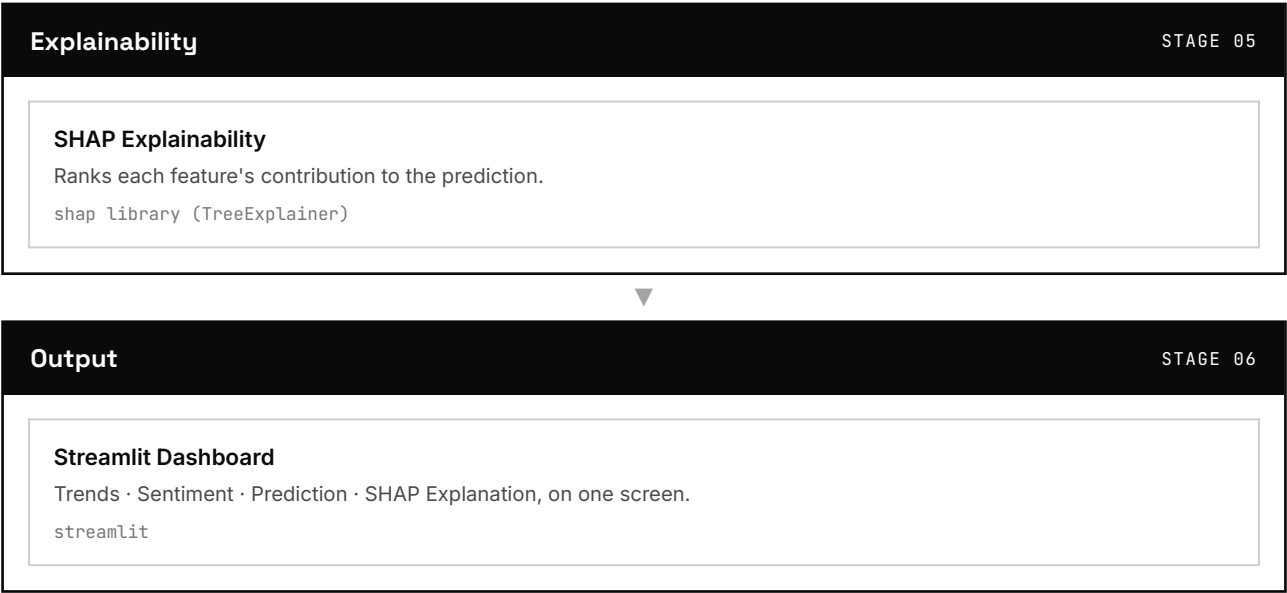


FIG. 2.1 – STOCKSENSE SYSTEM ARCHITECTURE, SIX STAGES / EIGHT COMPONENTS, MATCHING THE PIPELINE REFERENCED THROUGHOUT DOCUMENT 01.

03 Layered Architecture Breakdown

Eight layers, each with a single responsibility, a defined input/output contract, and one owning technology.

1

DATA COLLECTION LAYER

In: ticker symbol, date range · **Out:** raw OHLCV DataFrame + raw headline records · **Owner:** yfinance, News API / scraper

Pulls 5 years of daily OHLCV per ticker via yfinance, and daily financial news headlines via a News API or scheduled scraper. This is the only layer that talks to external services — every downstream layer works on data already materialised to disk or memory.

2

PREPROCESSING & FEATURE ENGINEERING LAYER

In: raw OHLCV · **Out:** cleaned OHLCV + technical indicator columns · **Owner:** pandas, pandas-ta

Cleans missing sessions, split/adjustment artefacts, and outliers; computes RSI, SMA/EMA, and rolling Volatility. Also the layer where EDA (distribution checks, correlation analysis) is performed before features are finalised.

3

SENTIMENT ANALYSIS LAYER

In: raw headline text · **Out:** daily FinBERT + VADER sentiment scores · **Owner:** HuggingFace Transformers, vaderSentiment

Runs independently of the price pipeline. FinBERT produces a probability-weighted sentiment class (positive / negative / neutral) per headline, aggregated to a daily score; VADER produces a compound score per headline on the same aggregation window.

4

SEQUENCE MODELING LAYER

In: windowed, normalised OHLCV · **Out:** learned sequence embedding per day · **Owner:** TensorFlow / Keras

The LSTM network (full spec in Section 05) consumes fixed-length windows of normalised OHLCV and technical-indicator values, and produces either a learned embedding or a price-direction probability that becomes one input to the fusion layer.

5

FEATURE FUSION LAYER

In: LSTM output + indicators + sentiment scores · **Out:** one fused feature vector per trading day · **Owner:** pandas / NumPy

A pure concatenation-and-alignment layer — no learning happens here. Its only job is to guarantee every daily row is time-aligned and schema-consistent (Section 07) before it reaches the classifier.

6

PREDICTION LAYER

In: fused feature vector · **Out:** UP/DOWN class + probability · **Owner:** XGBoost (scikit-learn API)

A trained gradient-boosted tree ensemble (full spec in Section 06) makes the final binary call and exposes a class probability used both as the dashboard's "confidence" figure and as SHAP's baseline for explanation.

7

EXPLAINABILITY LAYER

In: trained XGBoost model + fused feature vector · **Out:** per-feature SHAP values · **Owner:** shap (TreeExplainer)

Runs SHAP's TreeExplainer against the trained XGBoost model to compute exact Shapley values per feature per prediction — the fastest and most exact SHAP mode available for tree ensembles, avoiding the approximation overhead of KernelExplainer.

8

PRESENTATION LAYER

In: prediction, SHAP values, price/sentiment series · **Out:** rendered dashboard · **Owner:** Streamlit

Purely a rendering layer per the design system (Document 02) — it holds no modelling logic, only visual composition of what layers 1–7 have already computed.

04 Data Flow — Sequence Walkthrough

What happens, in order, from the moment a user opens the dashboard to the moment they see an explained prediction.

T+0.0s	→	User selects ticker & date range Streamlit sidebar control triggers a pipeline run for the selected ticker.
T+0.1s	→	Data Collection Layer fetches OHLCV + headlines yfinance returns cached or fresh OHLCV; headline source returns the latest daily batch.
T+0.4s	→	Preprocessing computes technical indicators RSI, SMA/EMA, Volatility calculated over the returned OHLCV window.
T+0.6s	→	Sentiment Layer scores headlines FinBERT and VADER independently score the day's headlines; scores aggregated to one row per day.
T+1.2s	→	LSTM produces a sequence embedding The trained LSTM runs inference on the latest input window (pre-trained weights loaded, not retrained per request).
T+1.3s	→	Fusion layer builds the feature vector LSTM embedding + indicators + sentiment scores concatenated per the schema in Section 07.
T+1.4s	→	XGBoost predicts UP / DOWN Trained classifier scores the fused vector and returns a class + probability.
T+1.6s	→	SHAP computes per-feature contributions TreeExplainer runs against the same fused vector and trained model.
T+1.8s	→	Dashboard renders Trends, sentiment series, prediction card, and SHAP panel all update on screen (Document 02, Section 08).

Timings are illustrative for a single-ticker CPU inference run — the LSTM and XGBoost models are pre-trained and only run inference at request time; training happens offline (Section 12).

05 LSTM Model Architecture

The proposed reference specification for the sequence-modelling network — sized to train comfortably on the CPU-class hardware specified in Document 01, Section 14.

PARAMETER	SPECIFICATION
Input window	60 trading days of OHLCV + technical-indicator features per sample (sliding window).
Input shape	(batch, 60, n_features)
Layer 1	LSTM(64, return_sequences=True)
Layer 2	Dropout(0.2)
Layer 3	LSTM(32, return_sequences=False)
Layer 4	Dropout(0.2)
Output head	Dense(16, activation="relu")
Embedding output	The 16-unit dense output is taken as the LSTM's learned representation and passed into the Feature Fusion layer — not a final prediction on its own.
Optimizer	Adam, lr=1e-3
Loss	Binary cross-entropy (during standalone pretraining on next-day direction, before the embedding is repurposed for fusion).
Normalisation	Min-max or z-score normalisation per feature, fit on the training window only, to avoid look-ahead bias.
Train / validation split	Chronological split (not random shuffling) — earlier years train, most recent period validates, to respect time-series causality.

DESIGN RATIONALE

A two-layer LSTM (64 → 32 units) balances enough capacity to capture multi-week momentum patterns against the overfitting risk of a deeper network on a single-ticker, 5-year dataset. Dropout of 0.2 after each recurrent layer is standard practice for regularising LSTMs on financial time series, where the signal-to-noise ratio is low.

06 XGBoost Classifier Configuration

The reference hyperparameter specification for the fusion-stage classifier that makes the final UP/DOWN call.

PARAMETER	SPECIFICATION
Objective	binary:logistic
Eval metric	logloss, auc
n_estimators	300 (with early stopping)
max_depth	4-6
learning_rate	0.05
subsample	0.8
colsample_bytree	0.8
early_stopping_rounds	20, monitored on chronological validation split
Output	Predicted class (UP=1 / DOWN=0) and predicted probability, the latter surfaced as "confidence" on the dashboard.

6.1 Why these defaults

Shallow trees (depth 4-6) and column/row subsampling keep the ensemble from memorising noise in a relatively small, single-ticker daily dataset. A moderate learning rate with early stopping on a chronological validation split lets `n_estimators` be set generously without manually tuning the exact boosting round count — the model stops itself once validation log-loss stops improving.

6.2 Why XGBoost is also the explainability anchor

Because the final decision is a tree ensemble rather than a neural network, SHAP's `TreeExplainer` can compute exact Shapley values in polynomial time instead of the sampling-based approximation required for arbitrary models — which is the specific technical reason the architecture routes the final decision through XGBoost rather than ending the pipeline at the LSTM.

07 Fused Feature Schema

The exact per-day feature vector that leaves the Fusion layer and enters the XGBoost classifier — this schema is the contract every upstream layer must satisfy.

FEATURE	TYPE	SOURCE LAYER	DESCRIPTION
lstm_embed_0...15	float ×16	Sequence Modeling	16-dimensional learned representation from the LSTM's final dense layer.
rsi_14	float	Preprocessing	14-day Relative Strength Index.
sma_20 / ema_20	float	Preprocessing	20-day simple / exponential moving average.
volatility_10d	float	Preprocessing	10-day rolling standard deviation of returns.
volume_change	float	Preprocessing	Day-over-day traded volume percentage change.
finbert_sentiment	float [-1, 1]	Sentiment Analysis	Aggregated daily FinBERT sentiment score.
vader_sentiment	float [-1, 1]	Sentiment Analysis	Aggregated daily VADER compound score.
headline_count	int	Sentiment Analysis	Number of headlines aggregated into that day's sentiment score.
target_direction	{0,1}	Label (training only)	1 if next-day close > today's close, else 0 — not present at inference time.

All continuous features are normalised using statistics fit on the training window only, mirroring the LSTM's normalisation rule in Section 05, to avoid look-ahead bias at both stages.

08 Module & Folder Structure

One folder per architectural layer — the file tree is a direct map of Section 03.

```

stocksense/
├─ data/
│   ├─ raw/           # untouched yfinance + news pulls
│   └─ processed/     # cleaned OHLCV + indicators
├─ collection/
│   ├─ price_fetcher.py # yfinance wrapper
│   └─ news_fetcher.py  # News API / scraper
├─ preprocessing/
│   ├─ clean.py
│   └─ indicators.py    # RSI, SMA/EMA, Volatility
├─ sentiment/
│   ├─ finbert_scorer.py
│   └─ vader_scorer.py
├─ modeling/
│   ├─ lstm_model.py   # Section 05 spec
│   ├─ fusion.py       # feature concatenation
│   └─ xgb_classifier.py # Section 06 spec
├─ explainability/
│   └─ shap_explainer.py
├─ dashboard/
│   ├─ app.py          # Streamlit entrypoint
│   └─ components/
├─ models/             # saved .h5 / .json weights
├─ notebooks/          # EDA, experimentation
├─ tests/
├─ requirements.txt
└─ README.md

```

09 Interface Contracts

The function-level contract each layer exposes to the one after it — this is what keeps the pipeline swappable layer-by-layer.

```
fetch_price_history(ticker: str, years: int = 5) → pd.DataFrame
```

Returns raw OHLCV indexed by date. Sole responsibility of `collection/price_fetcher.py`.

```
compute_indicators(ohlc: pd.DataFrame) → pd.DataFrame
```

Adds `rsi_14`, `sma_20`, `ema_20`, `volatility_10d`, `volume_change` columns. Pure function — no I/O.

```
score_sentiment(headlines: list[str]) → dict
```

Returns `{"finbert_sentiment": float, "vader_sentiment": float, "headline_count": int}` for one day's headline batch.

```
lstm_embed(window: np.ndarray) → np.ndarray # shape (16,)
```

Runs inference on a pre-trained LSTM (Section 05) over a 60-day input window and returns the 16-dim embedding.

```
fuse_features(embed, indicators, sentiment) → pd.Series
```

Concatenates all three inputs per the schema in Section 07. Raises on any schema mismatch — this is the pipeline's single validation checkpoint.

```
predict(fused_vector: pd.Series) → tuple[str, float]
```

Returns `("UP" | "DOWN", probability)` from the trained XGBoost model.

```
explain(fused_vector: pd.Series) → list[dict]
```

Returns a ranked list of `{"feature": str, "shap_value": float}`, sorted by `|shap_value|` descending, for the SHAP panel.

10 Deployment Architecture

A single-process, CPU-only deployment, matching the hardware footprint specified in Document 01.

ASPECT	SPECIFICATION
Runtime	Single Python process running <code>streamlit run dashboard/app.py</code> .
Model artefacts	Pre-trained LSTM (.h5/SavedModel) and XGBoost (.json) weights loaded once at process start, cached in memory for the session.
Compute	CPU-only inference; no GPU required (Document 01, Section 14).
Data refresh	Price and news data re-fetched on demand per ticker selection; daily news ideally refreshed via a scheduled job rather than per page-load.
Caching	Streamlit's <code>@st.cache_data</code> / <code>@st.cache_resource</code> used to avoid re-fetching OHLCV or re-loading model weights on every widget interaction.
Environment	Reproducible via <code>requirements.txt</code> ; Python virtual environment or Docker container for consistent TensorFlow/XGBoost versions.

10.1 Why no separate backend service

Because Streamlit runs the same Python process that hosts the models, there is no separate API server or network hop between prediction and presentation — which keeps the architecture matched to the project's actual scale (single-ticker, on-demand, small user base) rather than over-engineering a client-server split that Section 13 shows isn't yet needed.

11 Technology Stack Matrix

Every layer mapped to its exact owning library, cross-referenced against Section 03.

LAYER	PRIMARY TECHNOLOGY	SUPPORTING LIBRARIES
1 · Data Collection	yfinance	News API client / requests + BeautifulSoup (scraper)
2 · Preprocessing	pandas	pandas-ta, NumPy
3 · Sentiment Analysis	FinBERT (HuggingFace)	transformers, vaderSentiment
4 · Sequence Modeling	TensorFlow / Keras	scikit-learn (scaling/splits)
5 · Feature Fusion	pandas / NumPy	—
6 · Prediction	XGBoost	scikit-learn API wrapper
7 · Explainability	SHAP	matplotlib (static SHAP plots, optional)
8 · Presentation	Streamlit	Matplotlib / native Streamlit charting

12 Error Handling & Retraining

12.1 Runtime error handling

FAILURE MODE	HANDLING
yfinance request fails / empty response	Fall back to last successfully cached OHLCV for that ticker; surface a "data may be stale" notice on the dashboard rather than crashing.
No headlines for the day	Sentiment defaults to the prior day's score with <code>headline_count = 0</code> , flagged in the SHAP panel so the user knows sentiment is stale, not wrong.
Feature schema mismatch at fusion	Fusion layer raises a validation error before reaching XGBoost — the pipeline fails loudly rather than serving a prediction on malformed input.
Model artefact missing/corrupted	Dashboard fails to start with a clear artefact-path error, rather than silently falling back to an untrained model.

12.2 Retraining pipeline

Both the LSTM and XGBoost models are trained offline, not per-request. A retraining run repeats Sections 05–06 end-to-end on an updated historical window, validates on a held-out chronological slice, and only replaces the production model artefact if validation accuracy meets or exceeds the currently deployed model's — a simple gate against silent regressions as market regimes shift (Document 01, Section 16).

13 Scalability & Performance

- **Current scope** — single-ticker, on-demand inference is well within CPU-only, single-process capacity; no horizontal scaling is needed at this scale.
- **Multi-ticker growth path** — the fusion schema (Section 07) is ticker-agnostic, so extending to a ticker universe means parameterising Stage 01-02 by ticker and batching inference, not redesigning the pipeline.
- **Bottleneck ranking** — FinBERT inference (a transformer forward pass) is the most compute-heavy step per request; batching headline scoring and caching same-day results is the first optimisation if request volume grows.
- **Model retraining cost** is separate from serving cost — retraining can run on a schedule (e.g. weekly) independent of dashboard responsiveness.

14 Security & Data Considerations

- All data sources (yfinance, news API) are public market/news data — no personally identifiable or account-linked data is collected or stored.
- API keys for any news source are held as environment variables, never committed to the repository.
- The dashboard is a decision-support tool, not a trading executor — it has no brokerage integration and cannot place trades, which removes an entire class of financial-transaction security risk from the system boundary.
- Model artefacts and cached data are treated as disposable/reproducible build outputs, not sensitive assets — they can be regenerated from source data and training code.

15 Appendix — Full File Tree

Expanded reference tree including test and notebook scaffolding, for implementation.

```

stocksense/
├─ data/
│  └─ raw/
│     │ └─ prices/{ticker}.csv
│     └─ news/{ticker}_{date}.json
│  └─ processed/
│     └─ {ticker}_features.parquet
├─ collection/
│  └─ price_fetcher.py
│  └─ news_fetcher.py
│  └─ __init__.py
├─ preprocessing/
│  └─ clean.py
│  └─ indicators.py
│  └─ __init__.py
├─ sentiment/
│  └─ finbert_scorer.py
│  └─ vader_scorer.py
│  └─ __init__.py
├─ modeling/
│  └─ lstm_model.py
│  └─ fusion.py
│  └─ xgb_classifier.py
│  └─ __init__.py
├─ explainability/
│  └─ shap_explainer.py
│  └─ __init__.py
├─ dashboard/
│  └─ app.py
│  └─ components/
│     │ └─ trend_panel.py
│     │ └─ sentiment_panel.py
│     │ └─ prediction_card.py
│     └─ shap_panel.py
│  └─ theme.py          # tokens from Document 02
├─ models/
│  └─ lstm_v1.h5
│  └─ xgb_v1.json
├─ notebooks/
│  └─ 01_eda.ipynb
│  └─ 02_model_experiments.ipynb

```

```
├─ tests/  
│   ├── test_indicators.py  
│   ├── test_fusion.py  
│   └─ test_pipeline_e2e.py  
├─ requirements.txt  
├─ .env.example  
└─ README.md
```

This tree is the implementation-ready counterpart to the layered architecture in Section 03 and the module contracts in Section 09.